
SVARA (Sistem Verifikasi & Administrasi Data Pengguna)

Kelompok:

- Mutiara Rizky Salsabila (41524010092)
- Deswita Nindya Putri (41524010082)
- Keisya Rizkia Kamila (41524010093)

1. Ringkasan Proyek

SVARA (Sistem Verifikasi & Administrasi Data Pengguna) adalah aplikasi web full-stack yang dibangun menggunakan Spring Boot 3.4.5 dan Java 17 di sisi backend, dengan antarmuka statis berbasis HTML5 dan Tailwind CSS (via CDN) di sisi frontend. Proyek ini disusun sebagai tugas akademik mata kuliah Pemrograman Berorientasi Objek untuk mendemonstrasikan arsitektur client-server, REST API, dan konektivitas ke basis data cloud.

Sistem ini memisahkan dua peran pengguna, yaitu Admin dan User, yang masing-masing memiliki tabel data dan alur login tersendiri. Selain fitur autentikasi dan manajemen data pengguna (CRUD), sistem juga mencatat transaksi pembayaran tiket konser melalui entitas Pembayaran.

Backend telah di-deploy secara publik pada platform Railway, sedangkan basis data PostgreSQL dikelola melalui Supabase. Seluruh halaman frontend (.html) turut disajikan langsung oleh Spring Boot dari folder static, sehingga aplikasi dapat diakses langsung dari browser tanpa server tambahan.

2. Fitur Utama

- **Registrasi & Login User:** Pendaftaran akun baru (POST /register) dan proses login (POST /user/login) berdasarkan pencocokan email dan password pada tabel User.
- **Login Admin Terpisah:** Autentikasi khusus Admin (POST /admin/login) yang divalidasi terhadap tabel Admin yang berbeda dari tabel User.
- **Manajemen Data User (CRUD):** Admin dapat melihat seluruh pengguna (GET /users), melihat detail (GET /users/{id}), memperbarui data (PUT /users/{id}), dan menghapus data (DELETE /users/{id}).
- **Pencatatan Pembayaran:** Penyimpanan transaksi pembelian tiket konser (POST /pembayaran) lengkap dengan nama konser, kategori tiket, jumlah tiket, metode pembayaran, dan total bayar; waktu transaksi dicatat otomatis oleh server dalam format tanggal Bahasa Indonesia.
- **Riwayat Pembelian per User:** Pengambilan riwayat transaksi berdasarkan nama pengguna (GET /pembayaran/riwayat?namaUser=...), diurutkan dari transaksi terbaru.
- **Proteksi Halaman di Sisi Klien:** Halaman Admin.html memeriksa status sessionStorage (isAdminLoggedIn) saat dimuat dan mengalihkan pengguna yang belum login ke halaman LoginAdmin.html.

3. Profil Pengguna & Peran

Berbeda dari sistem RBAC (Role-Based Access Control) penuh yang menggunakan token dan filter keamanan di backend, SVARA membedakan peran Admin dan User murni melalui pemisahan tabel data dan alur login yang berbeda. Tidak ditemukan mekanisme otorisasi (authorization) di lapisan backend penjelasan lebih lanjut ada pada bagian 8 (Standar Keamanan Sistem).

- **User (Pelanggan):** Melakukan registrasi dan login sendiri, melihat/memperbarui data profil, serta melakukan dan melihat riwayat transaksi pembayaran tiket miliknya.
- **Admin (Pengelola):** Login melalui alur terpisah, memiliki akses konseptual untuk melihat seluruh data user melalui dashboard (Admin.html) serta melakukan operasi Update dan Delete terhadap data user.

4. Teknologi yang Digunakan

Komponen	Teknologi
Backend	Spring Boot 3.4.5, Java 17
Web Layer	spring-boot-starter-web
Persistensi	spring-boot-starter-data-jpa (Hibernate)
Database	PostgreSQL, di-hosting melalui Supabase (connection pooler)
Driver DB	org.postgresql:postgresql (runtime)
Frontend	HTML5, Tailwind CSS (via CDN), Google Material Symbols
Interaksi Data	Fetch API (JavaScript) dengan format JSON
Build Tool	Maven (Maven Wrapper / mvnw)
Deployment	Railway (backend + halaman statis)

5. Arsitektur Sistem

SVARA menggunakan pola desain berlapis Controller – Service – Repository yang umum pada aplikasi Spring Boot, dikombinasikan dengan komunikasi client-server berbasis REST API dan format JSON.

A. Alur Komunikasi Client-Server

- Frontend (halaman .html) mengirim permintaan HTTP (GET/POST/PUT/DELETE) ke backend menggunakan Fetch API, berisi Header dan Body dalam format JSON.
- Permintaan diterima langsung oleh Svara2Controller tanpa melalui lapisan filter keamanan (tidak ada dependency Spring Security pada proyek ini).
- Controller mendelegasikan logika bisnis ke lapisan Service yang sesuai (AdminService, UserService, atau PembayaranService).
- Service berkomunikasi dengan Repository (Spring Data JPA) untuk membaca/menulis data ke PostgreSQL, lalu hasilnya dirakit kembali oleh Controller menjadi HTTP Response (Status Code + JSON Body).

B. Lapisan Arsitektur Backend

- **Model (Entity):** Representasi tabel database sebagai objek Java (Admin, User, Pembayaran) menggunakan anotasi JPA seperti `@Entity`, `@Id`, dan `@GeneratedValue`.
- **Repository:** Antarmuka `JpaRepository` yang menyediakan operasi CRUD standar, ditambah kueri turunan (derived query) seperti `findByEmailAndPassword` dan `findByNamaUserOrderByIdDesc`.
- **Service:** Berisi logika bisnis, seperti pencocokan kredensial login dan pencatatan tanggal transaksi pembayaran secara otomatis menggunakan `SimpleDateFormat` berlokal Indonesia.
- **Controller:** Satu kelas `Svara2Controller` menangani seluruh endpoint REST API proyek ini, diberi anotasi `@CrossOrigin("*")` sehingga dapat diakses dari origin manapun.

C. Arsitektur Frontend

- Halaman dibangun sebagai berkas HTML statis (bukan SPA framework) yang disajikan langsung oleh Spring Boot dari folder `src/main/resources/static`.
- Styling menggunakan Tailwind CSS yang dimuat melalui CDN, dengan konfigurasi tema kustom (warna, radius, tipografi) didefinisikan pada berkas `js/svara.js`.
- Interaksi ke backend (login, CRUD, pembayaran) dilakukan secara asynchronous melalui `fetch()` ke endpoint REST API, tanpa proses build/bundling terpisah.

6. Struktur Proyek

```
src/main/java/com/svara2/svara2/
├── Svara2Application.java
├── controller/
│   └── Svara2Controller.java    # Seluruh REST endpoint
├── Model/
│   ├── Admin.java
│   ├── Pembayaran.java
│   └── User.java
├── repository/
│   ├── AdminRepository.java
│   ├── PembayaranRepository.java
│   └── UserRepository.java
└── service/
    ├── AdminService.java
    ├── PembayaranService.java
    └── UserService.java

src/main/resources/
├── application.properties
└── static/
```

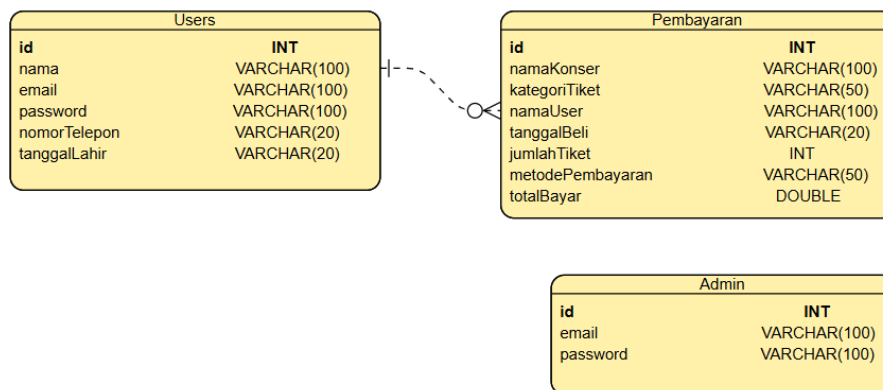
```

|— login.html
|— LoginAdmin.html
|— LoginUser.html
|— HomePage.html
|— Admin.html      # Dashboard admin (dilindungi sessionStorage)
|— Verifikasi.html
|— Pembayaran.html
|— Riwayat.html
|— js/svara.js      # Konfigurasi tema Tailwind bersama

```

7. Desain Database & Entitas Utama

Basis data SVARA terdiri atas tiga entitas independen. Berbeda dari desain berelasi (foreign key) pada umumnya, ketiga entitas ini tidak saling terhubung melalui relasi JPA (@ManyToOne / @OneToMany) keterkaitan antar data (misalnya Pembayaran ke User) hanya direpresentasikan sebagai teks bebas.



- **User:** Menyimpan data akun pelanggan (nama, email, password, nomor telepon, tanggal lahir) untuk keperluan registrasi dan login.
- **Admin:** Tabel terpisah untuk kredensial pengelola sistem (email, password), tidak memiliki relasi ke tabel lain.
- **Pembayaran:** Mencatat transaksi pembelian tiket konser. Field `namaUser` bertipe String biasa (bukan foreign key ke `User.id`), sehingga keterkaitan dengan data User bergantung pada kecocokan nilai teks, bukan relasi database yang ditegakkan (enforced).

8. Standar Keamanan Sistem

Berdasarkan tinjauan kode sumber, proyek ini belum menerapkan sejumlah praktik keamanan standar yang lazim digunakan pada aplikasi produksi. Bagian ini disusun secara transparan agar dapat menjadi bahan evaluasi/pengembangan lanjutan, mengingat proyek berstatus akademik.

- **Password disimpan dalam bentuk plain text:** Query login (findByEmailAndPassword) membandingkan password secara langsung tanpa hashing (misalnya BCrypt), baik pada tabel User maupun Admin.
- **Tidak ada lapisan keamanan backend:** pom.xml tidak menyertakan dependency Spring Security, sehingga tidak terdapat filter otentikasi (setara JwtFilter) maupun otorisasi berbasis role di sisi server.
- **CORS terbuka penuh:** Anotasi `@CrossOrigin("*")` pada `Svara2Controller` mengizinkan permintaan API dari domain manapun.
- **Proteksi halaman hanya di sisi klien:** Pembatasan akses ke Admin.html sepenuhnya bergantung pada pemeriksaan `sessionStorage.getItem("isAdminLoggedIn")` melalui JavaScript, yang dapat dilewati apabila endpoint API dipanggil langsung (misalnya melalui Postman atau cURL) tanpa melalui antarmuka login.
- **Kredensial database sebagai nilai default:** `application.properties` menyertakan nilai default untuk koneksi Supabase (URL, username, password) yang dapat ditimpa melalui environment variable. Untuk lingkungan produksi, kredensial sebaiknya sepenuhnya disuntikkan melalui environment variable pada platform deployment (mis. tab Variables di Railway), bukan disimpan sebagai default di dalam kode sumber.
- **Penanganan error generik:** Respons kegagalan login mengembalikan status 401 dengan pesan singkat ("Email atau password salah"); belum ditemukan penanganan exception terpusat (global exception handler) pada proyek ini.

9. Panduan Instalasi dan Menjalankan Aplikasi

Prasyarat: Java 17+, Maven (atau gunakan Maven Wrapper bawaan proyek), serta akses ke basis data PostgreSQL (Supabase atau instance lokal).

Tahap Pengaturan Lokal

- Clone repository proyek, lalu masuk ke direktori `svara2`.
- (Opsional) Atur koneksi database melalui environment variable agar tidak menggunakan nilai default:

```
export
SPRING_DATASOURCE_URL=jdbc:postgresql://<host>:5432/postgres
export SPRING_DATASOURCE_USERNAME=<username> export
SPRING_DATASOURCE_PASSWORD=<password>
```

Menjalankan Aplikasi

- **Linux/Mac:** `./mvnw spring-boot:run`
- **Windows (PowerShell/CMD):** `mvnw.cmd spring-boot:run`

Aplikasi berjalan pada port 9999 sesuai konfigurasi server.port di application.properties, sehingga dapat diakses melalui <http://localhost:9999/login.html>. Struktur tabel database dibuat/diperbarui secara otomatis oleh Hibernate (spring.jpa.hibernate.ddl-auto=update).

Deployment

Versi live backend & frontend telah di-deploy pada platform Railway dan dapat diakses melalui:

<https://project-svara-production.up.railway.app>

10. Daftar Endpoint API Utama (Backend)

Seluruh endpoint di bawah ini didefinisikan pada Svara2Controller dan dapat diakses tanpa autentikasi di lapisan backend (lihat bagian 8). Kolom "Digunakan oleh" bersifat konseptual berdasarkan alur aplikasi pada frontend, bukan pembatasan teknis yang ditegakkan server.

Modul	Endpoint	Method	Deskripsi	Digunakan oleh
Auth	/register	POST	Registrasi akun User baru	User
Auth	/user/login	POST	Login User (cek email & password)	User
Auth	/admin/login	POST	Login Admin (cek email & password)	Admin
Users	/users	GET	Mengambil seluruh data User	Admin
Users	/users/{id}	GET	Mengambil data User berdasarkan ID	Admin / User
Users	/users/{id}	PUT	Memperbarui data User	Admin
Users	/users/{id}	DELETE	Menghapus data User	Admin
Pembayaran	/pembayaran	POST	Menyimpan transaksi pembayaran tiket	User
Pembayaran	/pembayaran/riwayat?namaUser=...	GET	Riwayat pembelian tiket milik satu user, terbaru dahulu	User

11. Direktori UI (Frontend Views)

Halaman-halaman berikut disajikan sebagai berkas statis dari folder src/main/resources/static dan dikonsumsi langsung oleh browser:

Halaman	Fungsi	Catatan
login.html	Pintu masuk awal / pemilihan jenis login	Publik
LoginUser.html	Form login untuk User	Publik
LoginAdmin.html	Form login untuk Admin	Publik
HomePage.html	Halaman utama setelah User login	Perlu login (client-side)
Admin.html	Dashboard Admin — daftar & CRUD data User	Dilindungi sessionStorage (isAdminLoggedIn)
Verifikasi.html	Halaman verifikasi/konfirmasi terkait transaksi	Perlu login (client-side)
Pembayaran.html	Form input transaksi pembelian tiket	Perlu login (client-side)
Riwayat.html	Menampilkan riwayat pembelian tiket User	Perlu login (client-side)